

# TCGA Pathology Dataset Collection Pipeline

## Overview

This notebook documents and walks through the **data collection pipeline** used to build the TCGA pathology patch dataset that powers our cancer classification models.

The dataset was sourced from the **Genomic Data Commons (GDC)** — the official NIH/NCI portal for The Cancer Genome Atlas (TCGA) — which hosts thousands of whole-slide images (WSIs) in **.SVS** format captured from real patient tissue samples across 21 cancer types.

---

## Infrastructure: 3 EC2 Instances in Parallel

Downloading and processing whole-slide images is extremely resource-intensive. A single **.SVS** file can range from **500 MB to 5 GB**, and each cancer type contains hundreds of slides. To handle this at scale, we provisioned **three AWS EC2 instances** running concurrently:

| Instance       | Role               | Cancer Types Assigned                                                                |
|----------------|--------------------|--------------------------------------------------------------------------------------|
| ec2-instance-1 | Download + extract | TCGA-BRCA, TCGA-LUAD, TCGA-LUSC, TCGA-COAD, TCGA-READ, TCGA-PRAD, TCGA-GBM, TCGA-LGG |
| ec2-instance-2 | Download + extract | TCGA-LIHC, TCGA-STAD, TCGA-BLCA, TCGA-THCA, TCGA-OV, TCGA-UCEC, TCGA-PAAD            |
| ec2-instance-3 | Download + extract | TCGA-HNSC, TCGA-CESC, TCGA-SARC, TCGA-ESCA, TCGA-SKCM, TCGA-DLBC                     |

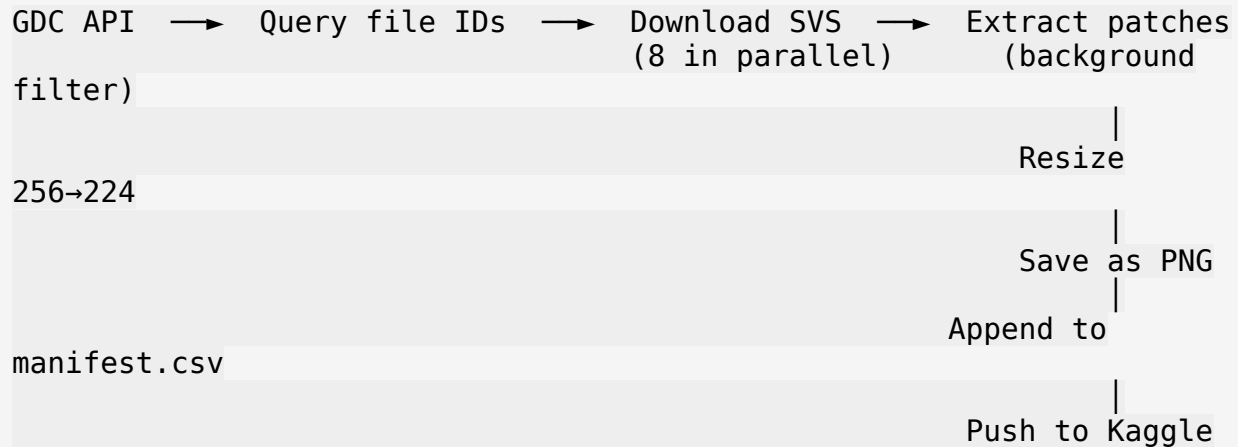
Each instance ran the **Download.py** script independently. Within each instance, **up to 8 SVS files were downloaded in parallel** using **ThreadPoolExecutor** to maximize network I/O throughput. After each slide finished downloading, patch extraction ran sequentially (OpenSlide is not thread-safe for concurrent reads on the same file).

Once each cancer type was fully processed, the patches were **automatically pushed to a Kaggle dataset** (ramachandraudupa/tcga-pathology-dataset) using the Kaggle CLI — so the data was incrementally available without waiting for all 3 instances to finish.

The pipeline supports **full crash recovery**: a **progress.json** file tracks which cancer types have been completed and pushed, so restarting the script on any instance picks up exactly where it left off.

---

## Pipeline Steps



## Dependencies

- `requests` — GDC REST API calls
- `openslide-python` — reads `.SVS` whole-slide images (requires `openslide-tools` system package)
- `Pillow` — image resizing and saving
- `numpy` — background detection
- `kaggle` — CLI for pushing results to Kaggle Datasets

```
import os
from typing import Optional
import sys
import csv
import json
import time
import threading
import requests
import numpy as np
import subprocess
import shutil
from pathlib import Path
from concurrent.futures import ThreadPoolExecutor, as_completed
from tqdm import tqdm
from collections import Counter

try:
    import openslide
    print("openslide-python available")
except ImportError:
    print("WARNING: openslide-python not installed. Run: pip install openslide-python")
```

```
from PIL import Image
print("All imports OK")
```

## Step 1 — Cancer Collections & GDC API Configuration

The GDC hosts TCGA data organized by **project ID** (e.g., TCGA-BRCA for breast cancer). We map each project ID to a human-readable label that becomes the folder name and the training class label.

We query only slides with:

- `data_format = SVS` — whole-slide image format
- `experimental_strategy = Tissue Slide` — H&E stained pathology slides (not genomics data)

```
# All TCGA cancer collections with histopathology (SM modality)
COLLECTIONS = {
    "TCGA-LIHC": "liver_hepatocellular",
    "TCGA-STAD": "stomach_adenocarcinoma",
    "TCGA-BLCA": "bladder_urothelial",
    "TCGA-THCA": "thyroid_carcinoma",
    "TCGA-OV": "ovarian_serous",
    "TCGA-UCEC": "endometrial_carcinoma",
    "TCGA-PAAD": "pancreatic_adenocarcinoma",
    "TCGA-HNSC": "head_neck_squamous",
    "TCGA-CESC": "cervical_cancer",
    "TCGA-SARC": "sarcoma",
    "TCGA-ESCA": "esophageal_carcinoma",
    "TCGA-SKCM": "skin_melanoma",
    "TCGA-DLBC": "lymphoma",
    "TCGA-MESO": "mesothelioma",
    "TCGA-UCS": "uterine_carcinosarcoma",
    "TCGA-ACC": "adrenocortical_carcinoma",
    "TCGA-PCPG": "pheochromocytoma",
    "TCGA-TGCT": "testicular_cancer",
    "TCGA-CHOL": "bile_duct_cancer",
    "TCGA-UVM": "uveal_melanoma",
    "TCGA-LAML": "acute_myeloid_leukemia",
}

GDC_API = "https://api.gdc.cancer.gov"

print(f"Total cancer collections: {len(COLLECTIONS)}")
for project_id, label in COLLECTIONS.items():
    print(f" {project_id:<12} -> {label}")
```

## Step 2 — Configuration Variables

| Parameter         | Value | Meaning                   |
|-------------------|-------|---------------------------|
| PATCHES_PER_SLIDE | 20    | Number of 256×256 patches |

| Parameter                           | Value | Meaning                                                                     |
|-------------------------------------|-------|-----------------------------------------------------------------------------|
| <code>PATCH_SIZE</code>             | 256   | randomly sampled per SVS<br>Raw crop size from the SVS at 20× magnification |
| <code>FINAL_SIZE</code>             | 224   | Resize target — matches ImageNet pretraining resolution                     |
| <code>MAX_PARALLEL_DOWNLOADS</code> | 8     | Concurrent SVS downloads per EC2 instance                                   |
| <code>KEEP_SVS</code>               | False | Delete the large SVS after extraction to save disk                          |

With 3 EC2 instances × 8 parallel downloads each, we had an effective throughput of **up to 24 concurrent SVS downloads** across the fleet.

```

OUT_DIR          = "/mnt/data/pathology_data"
MAX_FILES_PER_TYPE = 999          # download as many slides as GDC provides
PATCHES_PER_SLIDE = 20          # random patches per slide
PATCH_SIZE      = 256          # raw crop size at level 0 (20x magnification)
FINAL_SIZE       = 224          # resize to match ImageNet pretrain resolution
KEEP_SVS         = False        # delete SVS after extraction to save disk
MAX_PARALLEL_DOWNLOADS = 8      # per EC2 instance

KAGGLE_DATASET_SLUG = "ramachandraudupa/tcga-pathology-full"
KAGGLE_STAGING_DIR  = "/mnt/data/kaggle_staging"
KAGGLE_TMP_DIR      = "/mnt/data/kaggle_tmp" # keep zips off root FS to avoid ENOSPC

print("Configuration loaded.")

```

## Step 3 — Resume / Progress Tracking

Because downloading 21 cancer types takes many hours (sometimes days), we built **crash-safe resume logic**:

- `progress.json` stores the set of cancer types that have been fully processed **and** pushed to Kaggle.
- On restart, completed types are skipped entirely.
- Within a cancer type, already-extracted slides are detected by checking if their patch files exist.
- `manifest.csv` is written after every cancer type, so a crash never loses more than one type's worth of rows.

```
def load_progress(output_root: Path) -> dict:
    progress_file = output_root / "progress.json"
    if progress_file.exists():
        with open(progress_file) as f:
            data = json.load(f)
            data["completed"] = set(data.get("completed", []))
        return data
    return {"completed": set()}

def save_progress(output_root: Path, completed: set):
    progress_file = output_root / "progress.json"
    with open(progress_file, "w") as f:
        json.dump({"completed": sorted(completed)}, f, indent=2)

print("Resume helpers defined.")
```

## Step 4 — Querying the GDC API

We use the GDC REST API (`/files` endpoint) with a structured JSON filter to retrieve file IDs for all SVS tissue slides belonging to a given TCGA project.

The response gives us:

- `file_id` — unique GDC identifier used to construct the download URL
- `file_name` — original slide filename
- `cases.samples.sample_type` — whether the sample is **Primary Tumor**, **Solid Tissue Normal**, **Metastatic**, etc.

```
def get_file_ids(project_id: str, max_files: int = 5) -> list:
    """Query GDC API and return list of SVS file records for a
    project."""
    payload = {
        "filters": {
            "op": "and",
            "content": [
                {"op": "=", "content": {"field":
"cases.project.project_id", "value": project_id}},
                {"op": "=", "content": {"field": "data_format",
"value": "SVS"}},
                {"op": "=", "content": {"field":
"experimental_strategy", "value": "Tissue Slide"}}
            ]
        },
        "fields": "file_id,file_name,cases.samples.sample_type",
        "size": max_files,
        "format": "JSON"
    }
    try:
```

```

        resp = requests.post(f"{GDC_API}/files", json=payload,
                              timeout=(15, 120))
        resp.raise_for_status()
        hits = resp.json().get("data", {}).get("hits", [])
        return hits
    except Exception as e:
        print(f" [!] Failed to query {project_id}: {e}")
        return []

# Demo: query 3 slides from TCGA-LIHC (liver)
demo_files = get_file_ids("TCGA-LIHC", max_files=3)
print(f"Found {len(demo_files)} files for TCGA-LIHC")
for f in demo_files:
    sample_type = f.get("cases", [{}])[0].get("samples", [{}])
    [0].get("sample_type", "unknown")
    print(f" {f['file_name']} | sample_type={sample_type}")

```

## Step 5 — Parallel SVS Download

Each SVS file is downloaded directly from the GDC data portal at [https://api.gdc.cancer.gov/data/{file\\_id}](https://api.gdc.cancer.gov/data/{file_id}) using a streaming HTTP GET to avoid loading the full file into RAM.

**Thread safety:** each thread writes to its own file and uses a shared `threading.Lock` only for console output, so downloads never interfere with each other.

```

def download_svs(
    file_id: str,
    file_name: str,
    dest_dir: Path,
    print_lock: Optional[threading.Lock] = None,
) -> Optional[Path]:
    """Download one SVS file from GDC. Returns path if successful."""
    dest_path = dest_dir / file_name
    if dest_path.exists():
        print(f" [skip] {file_name} already downloaded")
        return dest_path

    url = f"{GDC_API}/data/{file_id}"
    try:
        resp = requests.get(url, stream=True, timeout=(30, 120))
        resp.raise_for_status()
        total = int(resp.headers.get("content-length", 0))
        with open(dest_path, "wb") as f, tqdm(
            desc=f" DL {file_name[:35]}", total=total,
            unit="B", unit_scale=True, leave=False,
        ) as bar:
            for chunk in resp.iter_content(chunk_size=1024 * 1024):
                f.write(chunk)

```

```

        bar.update(len(chunk))
    return dest_path
except Exception as e:
    print(f" [!] Download failed for {file_name}: {e}")
    if dest_path.exists():
        dest_path.unlink()
    return None

print("download_svs() defined.")

```

## Step 6 — Background Detection

Whole-slide images contain large regions of **white background** (glass slide area with no tissue). Randomly sampled patches often land on these blank regions.

We discard any patch where **more than 85% of pixels are near-white** ( $R > 200$ ,  $G > 200$ ,  $B > 200$ ). This filter ensures every saved patch contains actual tissue and improves training data quality.

```

def is_background(arr: np.ndarray, threshold: float = 0.85) -> bool:
    """Return True if patch is mostly white background."""
    white_mask = (arr[:, :, 0] > 200) & (arr[:, :, 1] > 200) &
    (arr[:, :, 2] > 200)
    white_ratio = white_mask.sum() / white_mask.size
    return white_ratio > threshold

# Demo: check a synthetic white patch
white_patch = np.full((256, 256, 3), 230, dtype=np.uint8)
tissue_patch = np.random.randint(0, 180, (256, 256, 3),
dtype=np.uint8)
print(f"White patch is background: {is_background(white_patch)}")
print(f"Tissue patch is background: {is_background(tissue_patch)}")

```

## Step 7 — Patch Extraction with OpenSlide

Once a slide is downloaded, we use **OpenSlide** to open it and randomly sample `PATCHES_PER_SLIDE` non-background  $256 \times 256$  patches from level 0 (highest magnification,  $\sim 20\times$ ).

Each patch is:

1. Read from the SVS at a random (x, y) coordinate
2. Converted from RGBA to RGB
3. Checked against the background filter
4. Resized from  $256 \times 256$  to  $224 \times 224$  using LANCZOS resampling
5. Saved as a PNG with the coordinate baked into the filename:  
`{slide_stem}_x{x}_y{y}.png`

The coordinate in the filename is later used to extract the **patient/case barcode** for patient-level train/val splitting (no data leakage).

```
def extract_patches(
    svs_path: Path,
    output_dir: Path,
    patch_size: int = 256,
    target_size: int = 224,
    patches_per_slide: int = 50,
    max_attempts_multiplier: int = 20
) -> list:
    """
    Open slide, randomly sample patches, filter background,
    resize, and save as PNG. Returns list of saved patch paths.
    """
    saved_paths = []
    try:
        slide = openslide.OpenSlide(str(svs_path))
    except Exception as e:
        print(f" [!] Cannot open slide {svs_path.name}: {e}")
        return saved_paths

    try:
        width, height = slide.dimensions
        if width < patch_size or height < patch_size:
            return saved_paths

        max_x, max_y = width - patch_size, height - patch_size
        max_attempts = patches_per_slide * max_attempts_multiplier
        attempts = saved = 0

        with tqdm(total=patches_per_slide, desc=f" Patches
{svs_path.name[:30]}", leave=False) as pbar:
            while saved < patches_per_slide and attempts <
max_attempts:
                attempts += 1
                x = int(np.random.randint(0, max(1, max_x)))
                y = int(np.random.randint(0, max(1, max_y)))
                try:
                    region = slide.read_region((x, y), 0, (patch_size,
patch_size)).convert("RGB")
                except Exception:
                    continue
                if is_background(np.array(region)):
                    continue
                if target_size != patch_size:
                    region = region.resize((target_size, target_size),
Image.LANCZOS)
                patch_name = f"{svs_path.stem}_x{x}_y{y}.png"
                region.save(output_dir / patch_name, "PNG",
```

```

optimize=True)
        saved_paths.append(output_dir / patch_name)
        saved += 1
        pbar.update(1)
    finally:
        slide.close()

    return saved_paths

print("extract_patches() defined.")

```

## Step 8 — Processing One Cancer Collection

The `process_collection()` function orchestrates the full pipeline for a single cancer type:

1. Query GDC for up to `MAX_FILES_PER_TYPE` slides
2. Skip slides already extracted (patch files on disk)
3. Submit all remaining downloads to a `ThreadPoolExecutor` (8 workers per instance)
4. As each download completes, extract patches sequentially from that slide
5. Return manifest rows (one row per patch)

After all slides are processed, the manifest is appended and the cancer type is marked complete in `progress.json`.

## Step 9 — Kaggle Dataset Upload

After all cancer types on an instance are processed, the full dataset (patch folders + `manifest.csv`) is pushed to Kaggle as a public dataset using the Kaggle CLI.

```

# First push: creates the dataset
kaggle datasets create -p /mnt/data/kaggle_staging --dir-mode zip

# Subsequent pushes: add a new version
kaggle datasets version -p /mnt/data/kaggle_staging -m "added
liver_hepatocellular" --dir-mode zip

```

The `KAGGLE_TMP_DIR` is set to `/mnt/data/` (the data volume, not the root filesystem) to avoid **[Errno 28] No space left on device** errors when the Kaggle CLI creates the temporary zip archive of a multi-GB dataset.

The resulting Kaggle dataset (`ramachandraudupa/tcga-pathology-dataset`) is consumed directly by the training scripts via `kaggle datasets download`.

## Dataset Summary

After running across 3 EC2 instances the final dataset contains:

| Property            | Value                                                           |
|---------------------|-----------------------------------------------------------------|
| Cancer types        | 21 TCGA collections                                             |
| Sample types        | Primary Tumor, Solid Tissue Normal, Recurrent Tumor, Metastatic |
| Patch size          | 224 × 224 pixels (RGB PNG)                                      |
| Magnification       | ~20× (level 0)                                                  |
| Patient-level split | Guaranteed by TCGA barcode in filename                          |
| Storage format      | Kaggle Dataset with <code>manifest.csv</code> index             |

The `manifest.csv` contains one row per patch:

```
patch_path, cancer_type, project_id, sample_type, source_slide,
file_id
```

This manifest is the entry point for all downstream training scripts.

## Main Entry Point

Run the full pipeline end-to-end. On each of the 3 EC2 instances, `CANCER_TYPES` was set to the subset assigned to that instance so they worked in parallel without overlap.

```
# This cell shows the main orchestration logic (read-only – do not run
on a local machine
# without a /mnt/data volume and Kaggle credentials configured).

def main_summary():
    """
    Pseudocode summary of main():

    1. Load progress.json → skip already-completed cancer types
    2. Load existing manifest.csv → deduplicate rows
    3. Check whether Kaggle dataset already exists (once, before the
    loop)
    4. For each remaining cancer type:
        a. process_collection() → parallel download + sequential
    extraction
        b. Append rows to manifest.csv
        c. Mark completed in progress.json
    5. push_all_to_kaggle() → move all folders to staging, push as
    new version
    6. Print per-cancer-type patch counts
    """
    pass
```

```
print("See Download.py for the full main() implementation.")  
print("Run on EC2 with: python Download.py")
```

# Hierarchical Dual-Head Cancer Classification

## TCGA Pathology — Multi-Task Learning with Cascaded Prediction

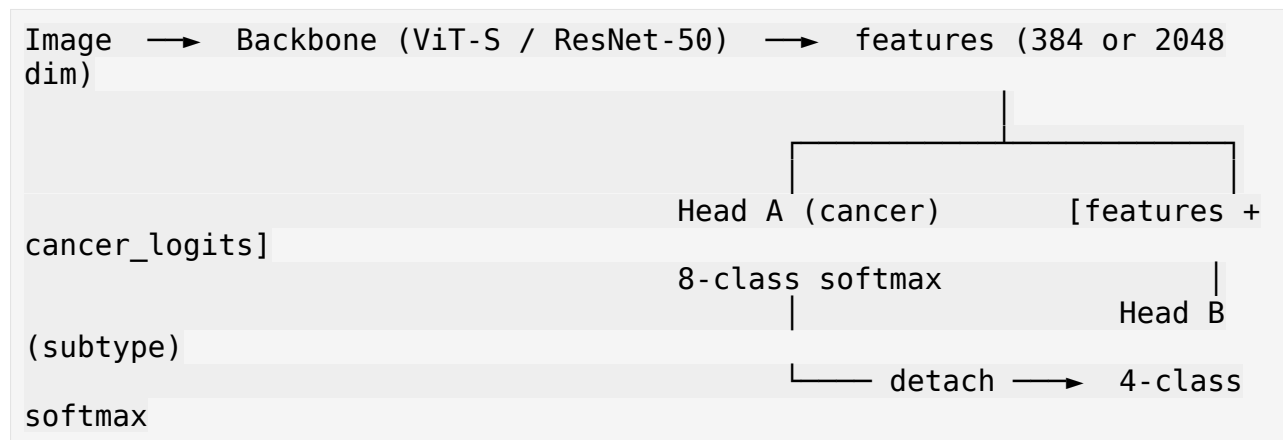
### Overview

This notebook covers our **first-generation** training approach for the TCGA pathology dataset: a **dual-head multi-task model** that simultaneously predicts:

1. **Head A — Cancer Type** (8 classes): *What kind of cancer is this?*  
breast\_cancer, lung\_adenocarcinoma, lung\_squamous\_cell,  
colon\_adenocarcinoma, rectal\_adenocarcinoma, prostate\_cancer,  
brain\_glioblastoma, brain\_lower\_grade\_glioma
2. **Head B — Sample Type** (4 classes): *What stage/subtype is this sample?*  
Primary Tumor, Solid Tissue Normal, Recurrent Tumor, Metastatic

### Key Innovation: Hierarchical Cascade

The subtype head is **not independent** — it receives the backbone features **concatenated with the cancer type prediction**. This implements a soft hierarchical constraint: *knowing the cancer type helps predict the subtype*.



The **detach()** on the cancer logits before they enter Head B is critical: it prevents the subtype loss from flowing back into Head A and corrupting the primary task's gradients.

### Combined Loss

$$\text{Loss} = 1.0 \times \text{CE}(\text{cancer\_type}) + 0.1 \times \text{CE}(\text{sample\_type})$$

Cancer type is weighted 10× higher because it is the clinically primary label and the harder task.

# Dataset Description

The dataset was collected from The Cancer Genome Atlas (TCGA) via the GDC portal using 3 parallel EC2 instances (see `Download_notebook.ipynb` for the full collection pipeline).

| Property         | Value                                     |
|------------------|-------------------------------------------|
| Total patches    | ~133,886                                  |
| Cancer types     | 8 (primary classification target)         |
| Sample subtypes  | 4 (secondary classification target)       |
| Patch resolution | 224 × 224 px (H&E stained, ~20×)          |
| Split strategy   | Patient-level GroupShuffleSplit (85/15)   |
| Data source      | TCGA via GDC API (SVS → patch extraction) |

## Why patient-level split?

Each patient contributes ~20 patches. If split randomly, the same patient's patches would appear in both train and validation, leaking patient-specific features (stain, scanner, tissue prep) and inflating validation accuracy.

## Dependencies & Imports

```
import os, sys, csv, json, time, logging, subprocess
from pathlib import Path

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.cuda.amp import GradScaler
from torch.amp import autocast
from torch.optim.swa_utils import AveragedModel, get_ema_multi_avg_fn
from torch.utils.data import Dataset, DataLoader,
WeightedRandomSampler
import torchvision.transforms as T
from sklearn.model_selection import GroupShuffleSplit
from sklearn.preprocessing import LabelEncoder
from tqdm import tqdm
import timm
from PIL import Image

print(f"PyTorch version : {torch.__version__}")
print(f"CUDA available : {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"GPU : {torch.cuda.get_device_name(0)}")
```

## Configuration

```
KAGGLE_DATASET_SLUG = "ramachandraudupa/tcga-pathology-dataset"
DATA_DIR             = Path("./data/tcga_train_data")
CHECKPOINT_DIR       = Path("./data/checkpoints")
LOG_FILE             = Path("./data/train.log")

MODELS_TO_TRAIN = ["vit_small", "resnet50"]

NUM_EPOCHS          = 50
BATCH_SIZE          = 512
NUM_WORKERS         = 16
LR                  = 1e-4
WEIGHT_DECAY        = 0.01
GRAD_CLIP           = 1.0
WARMUP_EPOCHS       = 5
LABEL_SMOOTHING     = 0.1
VAL_FRACTION        = 0.15
SEED                = 42
EMA_DECAY           = 0.9999

# Multi-task loss weights
# Cancer type is the primary clinical objective and harder task ->
# weighted 10x higher
LOSS_WEIGHT_CANCER  = 1.0
LOSS_WEIGHT_SUBTYPE = 0.1

RESUME = True
print("Configuration set.")
```

## Task 1 — Data Loading

The manifest is a CSV with one row per patch. Each row records the local path, cancer type, and sample type. We reconstruct the local path from just the filename and `cancer_type` column, because the manifest was generated on EC2 with a different absolute path.

```
def load_manifest(data_dir: Path):
    manifest_path = data_dir / "manifest.csv"
    if not manifest_path.exists():
        raise FileNotFoundError(f"manifest.csv not found in {data_dir}")

    paths, cancers, subtypes = [], [], []
    missing = 0

    with open(manifest_path, newline="") as f:
        for row in csv.DictReader(f):
            cancer = row.get("cancer_type", "").strip()
            subtype = row.get("sample_type", "").strip()
```

```

        if not cancer or not subtype:
            continue
        filename = Path(row["patch_path"]).name
        local_path = data_dir / "pathology_data" / cancer /
filename
        if not local_path.exists():
            missing += 1
            continue
        paths.append(str(local_path))
        cancers.append(cancer)
        subtypes.append(subtype)

    print(f"Loaded {len(paths)} valid samples, {missing} missing")
    return paths, cancers, subtypes

print("load_manifest() defined.")

```

## Task 2 — Dataset Class

Each item returns a triple: (image\_tensor, cancer\_label\_int, subtype\_label\_int). Both labels are needed for the dual-head loss.

```

IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD  = [0.229, 0.224, 0.225]

class PatchDataset(Dataset):
    def __init__(self, paths, cancer_labels, subtype_labels,
transform=None):
        self.paths          = paths
        self.cancer_labels  = cancer_labels
        self.subtype_labels = subtype_labels
        self.transform       = transform

    def __len__(self):
        return len(self.paths)

    def __getitem__(self, idx):
        img = Image.open(self.paths[idx]).convert("RGB")
        if self.transform:
            img = self.transform(img)
        return img, self.cancer_labels[idx], self.subtype_labels[idx]

def get_transforms(train: bool):
    if train:
        return T.Compose([
            T.RandomHorizontalFlip(),
            T.RandomVerticalFlip(),
            T.RandomRotation(90),

```

```

        T.ColorJitter(brightness=0.2, contrast=0.2,
saturation=0.2),
        T.Resize((224, 224)),
        T.ToTensor(),
        T.Normalize(IMAGENET_MEAN, IMAGENET_STD),
    ])
    return T.Compose([
        T.Resize((224, 224)),
        T.ToTensor(),
        T.Normalize(IMAGENET_MEAN, IMAGENET_STD),
    ])

print("PatchDataset and transforms defined.")

```

## Task 3 — Dual-Head Model Architecture

### Shared Backbone

We support two backbones loaded via `timm`, both pretrained on ImageNet-21k or ImageNet-1k:

| Backbone                           | Feature dim | Notes                                          |
|------------------------------------|-------------|------------------------------------------------|
| <code>vit_small_patch16_224</code> | 384         | ViT-S/16, self-attention on 16×16 patch tokens |
| <code>resnet50</code>              | 2048        | Classic CNN with global average pooling        |

### Cascaded Subtype Head

The subtype head takes `[features || cancer_logits]` as input — a vector of size `feat_dim + num_cancer`. This gives the subtype head direct access to the cancer prediction, implementing a **soft label hierarchy** without requiring a hard two-stage pipeline.

```

subtype_input = torch.cat([feat, cancer.detach()], dim=1)
subtype = self.head_subtype(subtype_input)

```

The `.detach()` is essential — without it, the subtype loss gradient would flow through `cancer` and back into the backbone, interfering with the primary cancer classification task.

```

class DualHeadModel(nn.Module):
    """
    Shared backbone → two classification heads.
    Head B (subtype) is conditioned on Head A (cancer) output –
    hierarchical cascade.
    """
    def __init__(self, backbone_name: str, num_cancer: int,

```

```

num_subtype: int):
    super().__init__()

    if backbone_name == "vit_small":
        self.backbone = timm.create_model("vit_small_patch16_224",
pretrained=True, num_classes=0)
        feat_dim = self.backbone.num_features # 384
    elif backbone_name == "resnet50":
        self.backbone = timm.create_model("resnet50",
pretrained=True, num_classes=0, global_pool="avg")
        feat_dim = self.backbone.num_features # 2048
    else:
        raise ValueError(f"Unknown backbone '{backbone_name}'")

    # Head A – primary task: cancer type classification
    self.head_cancer = nn.Sequential(
        nn.LayerNorm(feat_dim),
        nn.Dropout(0.3),
        nn.Linear(feat_dim, num_cancer),
    )

    # Head B – secondary task: sample subtype, conditioned on
cancer prediction
    # Input size = feat_dim + num_cancer (concatenation of
features and cancer logits)
    self.head_subtype = nn.Sequential(
        nn.LayerNorm(feat_dim + num_cancer),
        nn.Dropout(0.3),
        nn.Linear(feat_dim + num_cancer, num_subtype),
    )
    print(f"Model: {backbone_name} | feat_dim={feat_dim} | "
          f"cancer_classes={num_cancer} | "
subtype_classes={num_subtype}")

    def forward(self, x):
        feat = self.backbone(x) # (B,
feat_dim)
        cancer = self.head_cancer(feat) # (B,
num_cancer)
        # Detach cancer logits so subtype loss doesn't affect the
cancer head gradients
        subtype = self.head_subtype(torch.cat([feat, cancer.detach()],
dim=1))
        return cancer, subtype

# Demo: instantiate with 8 cancer classes and 4 subtypes
model = DualHeadModel("resnet50", num_cancer=8, num_subtype=4)
dummy = torch.randn(2, 3, 224, 224)
ca_out, st_out = model(dummy)

```

```
print(f"Cancer logits shape : {ca_out.shape}")
print(f"Subtype logits shape : {st_out.shape}")
```

## Task 4 — LR Schedule: Cosine Warmup

We use a **linear warmup** for the first 5 epochs followed by a **cosine annealing** decay. The warmup prevents the pretrained backbone weights from being destroyed by large gradients at the very start of fine-tuning.

```
def get_scheduler(optimizer, warmup_epochs, total_epochs,
steps_per_epoch):
    warmup_steps = warmup_epochs * steps_per_epoch
    total_steps = total_epochs * steps_per_epoch

    def lr_lambda(step):
        if step < warmup_steps:
            return float(step) / max(1, warmup_steps)
        progress = (step - warmup_steps) / max(1, total_steps -
warmup_steps)
        return max(0.0, 0.5 * (1.0 + np.cos(np.pi * progress)))

    return torch.optim.lr_scheduler.LambdaLR(optimizer, lr_lambda)

# Visualize the LR schedule
import matplotlib.pyplot as plt

dummy_opt = torch.optim.AdamW([torch.nn.Parameter(torch.randn(1))],
lr=1e-4)
sched = get_scheduler(dummy_opt, warmup_epochs=5, total_epochs=50,
steps_per_epoch=100)

lrs = []
for i in range(50 * 100):
    lrs.append(dummy_opt.param_groups[0]["lr"])
    sched.step()

plt.figure(figsize=(10, 3))
plt.plot([i/100 for i in range(len(lrs))], lrs)
plt.xlabel("Epoch"); plt.ylabel("LR"); plt.title("Cosine LR with
Linear Warmup")
plt.grid(True); plt.tight_layout(); plt.show()
```

## Task 5 — Training Loop

Each training step:

1. Forward pass → (cancer\_logits, subtype\_logits)
2. Compute weighted multi-task loss:  $1.0 \times \text{CE}_{\text{cancer}} + 0.1 \times \text{CE}_{\text{subtype}}$

3. Mixed-precision backward pass (AMP autocast + GradScaler)
4. Gradient clipping at 1.0 (prevents exploding gradients during early fine-tuning)
5. Optimizer step
6. Update EMA model weights

**Exponential Moving Average (EMA):** a shadow copy of the model whose weights are a decaying average of all past weights (decay=0.9999). EMA models consistently produce better validation accuracy than the live model and are used for the final best checkpoint.

```
def train_one_epoch(model, loader, criterion, optimizer, scheduler,
                    scaler,
                    device, epoch, total_epochs, ema_model=None):
    model.train()
    tot_loss = tot_ca_loss = tot_st_loss = 0.0
    ca_correct = st_correct = total = 0

    pbar = tqdm(loader, desc=f"E{epoch:02d}/{total_epochs} [Train]",
                 unit="batch", leave=False, dynamic_ncols=True)

    for imgs, ca_tgt, st_tgt in pbar:
        imgs = imgs.to(device, non_blocking=True)
        ca_tgt = ca_tgt.to(device, non_blocking=True)
        st_tgt = st_tgt.to(device, non_blocking=True)

        optimizer.zero_grad(set_to_none=True)

        with autocast(device_type="cuda"):
            ca_logits, st_logits = model(imgs)
            ca_loss = criterion(ca_logits, ca_tgt)
            st_loss = criterion(st_logits, st_tgt)
            loss = LOSS_WEIGHT_CANCER * ca_loss +
LOSS_WEIGHT_SUBTYPE * st_loss

        scaler.scale(loss).backward()
        scaler.unscale_(optimizer)
        torch.nn.utils.clip_grad_norm_(model.parameters(), GRAD_CLIP)
        scaler.step(optimizer)
        scaler.update()
        scheduler.step()

        if ema_model is not None:
            ema_model.update_parameters(model)

        bs = ca_tgt.size(0)
        tot_loss += loss.item() * bs
        tot_ca_loss += ca_loss.item() * bs
        tot_st_loss += st_loss.item() * bs
        ca_correct += (ca_logits.argmax(1) == ca_tgt).sum().item()
        st_correct += (st_logits.argmax(1) == st_tgt).sum().item()
        total += bs
```

```

pbar.set_postfix(
    loss=f"{tot_loss/total:.3f}",
    ca_acc=f"{ca_correct/total:.3f}",
    st_acc=f"{st_correct/total:.3f}",
)

return (tot_loss/total, tot_ca_loss/total, tot_st_loss/total,
        ca_correct/total, st_correct/total)

print("train_one_epoch() defined.")

```

## Task 6 — Checkpointing & Metrics

Three checkpoints are maintained per model:

| File                              | Contents                                                  | When saved                                    |
|-----------------------------------|-----------------------------------------------------------|-----------------------------------------------|
| <code>{model}_latest.pth</code>   | Full state (model + EMA + optimizer + scheduler + scaler) | Every epoch — for crash recovery              |
| <code>{model}_best.pth</code>     | Normal model weights                                      | When <code>val_cancer_acc</code> improves     |
| <code>{model}_ema_best.pth</code> | EMA model weights                                         | When <code>ema_val_cancer_acc</code> improves |

A CSV log (`{model}_metrics.csv`) is appended after every epoch with all loss and accuracy values for both heads on both the normal and EMA model. This makes it easy to plot training curves.

## Task 7 — Class Imbalance: WeightedRandomSampler

The TCGA dataset is **imbalanced** — some cancer types have many more patches than others. We address this with a `WeightedRandomSampler` on the training loader:

```

ca_counts      = np.bincount(tr_ca, minlength=num_cancer)
ca_weights     = 1.0 / np.maximum(ca_counts, 1)  # inverse-frequency
sample_weights = torch.tensor(ca_weights[tr_ca], dtype=torch.float)
sampler        = WeightedRandomSampler(sample_weights,
len(sample_weights), replacement=True)

```

Each epoch, rare classes are oversampled proportionally so the model sees an approximately balanced distribution during training, even though the underlying dataset is not.

## Task 8 — Training Curve Visualization

```

def plot_metrics(csv_path: Path, model_name: str):
    import matplotlib.pyplot as plt
    import pandas as pd

```

```

if not csv_path.exists():
    print(f"No metrics file found at {csv_path}")
    return

df = pd.read_csv(csv_path)
if len(df) == 0:
    return

fig, axes = plt.subplots(1, 2, figsize=(15, 6))

axes[0].plot(df["epoch"], df["tr_loss"], label="Train Loss")
axes[0].plot(df["epoch"], df["va_loss"], label="Val Loss")
axes[0].plot(df["epoch"], df["ema_va_loss"], label="EMA Val Loss",
linestyle="--")
axes[0].set_title("Loss Curves"); axes[0].set_xlabel("Epoch")
axes[0].legend(); axes[0].grid(True)

axes[1].plot(df["epoch"], df["tr_ca_acc"], label="Train Cancer
Acc")
axes[1].plot(df["epoch"], df["va_ca_acc"], label="Val Cancer
Acc")
axes[1].plot(df["epoch"], df["ema_va_ca_acc"], label="EMA Val
Cancer Acc", linestyle="--")
axes[1].set_title("Cancer Accuracy"); axes[1].set_xlabel("Epoch")
axes[1].legend(); axes[1].grid(True)

plt.suptitle(f"Training Curves – {model_name}", fontsize=14,
fontweight="bold")
plt.tight_layout()
plt.show()

# Example: load and plot an existing metrics CSV if available
for model_name in ["vit_small", "resnet50"]:
    csv_path = Path("./data/checkpoints") /
f"{model_name}_metrics.csv"
    plot_metrics(csv_path, model_name)

```

## Summary & Key Design Decisions

| Decision                                | Choice                  | Rationale                                                                     |
|-----------------------------------------|-------------------------|-------------------------------------------------------------------------------|
| Architecture                            | Dual-head with cascade  | Subtype is semantically dependent on cancer type — conditioning captures this |
| Loss weights                            | Cancer 1.0, Subtype 0.1 | Prevent subtype task from dominating the more important cancer classification |
| <code>.detach()</code> on cancer logits | Yes                     | Isolates gradient flows: subtype loss cannot corrupt cancer head training     |

| Decision           | Choice                          | Rationale                                                                          |
|--------------------|---------------------------------|------------------------------------------------------------------------------------|
| Imbalance handling | WeightedRandomSampler           | Rare cancer types are oversampled so the model doesn't ignore them                 |
| EMA                | decay=0.9999                    | EMA model consistently outperforms the live model on validation                    |
| Split              | Patient-level GroupShuffleSplit | Prevents data leakage across patients with the same tissue/scanner characteristics |
| Mixed precision    | AMP autocast + GradScaler       | ~2× speedup on A100/V100 with no accuracy loss                                     |

## Limitations of This Version

This dual-head model was our first experiment. While it correctly identifies the hierarchical relationship between cancer type and sample type, we observed:

- **Overfitting** (~92% train / ~75% val cancer accuracy) caused by cross-patient domain shift in staining and scanning protocols
- The flat concatenation of cancer logits into the subtype head is a soft constraint — a true hierarchical MIL approach would be stronger

These limitations motivated the improved approach in `Train2.py` (HED stain augmentation + class-weighted loss + slide-level evaluation).

# TCGA Fine-Grained Cancer Subtype Classification

## Transfer Learning with DINOv2 & ResNet-50 | HED Stain Augmentation | Slide-Level Evaluation

---

### Overview

This notebook documents our **second-generation** training pipeline for the TCGA histopathology dataset — an improved approach addressing the key weaknesses of the earlier dual-head model.

### Problem Statement

Classifying cancer subtypes from H&E-stained whole-slide image patches is a **hard fine-grained recognition problem**. Two patches from the same cancer type but different patients can look visually different due to:

- Staining protocol differences between hospitals
- Scanner hardware (Aperio vs Leica vs Hamamatsu)
- Tissue preparation artifacts

Our first model achieved ~92% training accuracy but only ~75% validation accuracy — a classic cross-patient domain shift / overfitting pattern.

### What's Different in This Version

| Change               | Old Model (rrr)                  | This Model (Train2)                                                   |
|----------------------|----------------------------------|-----------------------------------------------------------------------|
| Label scheme         | 2 separate heads (8 + 4 classes) | 1 flat label: <code>cancer_type_sample_type</code> (up to 32 classes) |
| Backbone             | ViT-S/16 (plain ImageNet)        | DINOv2 ViT-S/14 with register tokens + ResNet-50                      |
| LLRD                 | No                               | Yes — per-layer learning rate decay for ViT                           |
| Imbalance            | WeightedRandomSampler            | Class-weighted CrossEntropyLoss                                       |
| Stain augmentation   | ColorJitter only                 | <b>HED stain-space jitter</b> (Tellez et al. 2019)                    |
| Evaluation           | Per-patch accuracy               | <b>Per-patch + per-slide (case) accuracy</b>                          |
| Checkpoint selection | Best patch accuracy              | Best <b>slide-level</b> accuracy                                      |

# Dataset Description

## Collection Infrastructure

The dataset was collected from **The Cancer Genome Atlas (TCGA)** via the NIH Genomic Data Commons (GDC) API. Whole-slide images (.SVS format) were downloaded and processed across **3 AWS EC2 instances** running in parallel, each handling a distinct subset of the 21 TCGA cancer collections.

- **EC2 Instance 1:** BRCA, LUAD, LUSC, COAD, READ, PRAD, GBM, LGG
- **EC2 Instance 2:** LIHC, STAD, BLCA, THCA, OV, UCEC, PAAD
- **EC2 Instance 3:** HNSC, CESC, SARC, ESCA, SKCM, DLBC

Each instance used **8 parallel download threads** per cancer type and extracted  $20 \times 224\text{px}$  patches per slide at level 0 (~20× magnification), filtering blank/background patches. Completed cancer types were incrementally pushed to Kaggle to avoid data loss on instance interruption.

## Label Construction — Flat Labels

Rather than a two-head architecture, we **flatten** the cancer type and sample type into a single label:

```
"breast_cancer" + "Primary_Tumor" → "breast_cancer_Primary_Tumor"  
"rectal_adenocarcinoma" + "Metastatic" →  
"rectal_adenocarcinoma_Metastatic"
```

This gives up to 32 combined classes (only observed combinations appear). A single-head classifier on these flat labels is simpler, more stable, and avoids the cascading gradient issues of the dual-head design.

## Train / Validation Split

Patches are split **at the patient level** using `GroupShuffleSplit` with the TCGA barcode (TCGA-XX-XXXX) as the group key. This guarantees:

- No patient's patches appear in both train and val
- Validation measures generalization to **unseen patients**, not unseen patches from the same patient
- The reported accuracy reflects real-world deployment conditions

## Dependencies & Imports

```
import os, sys, csv, json, time, logging, subprocess, re  
from pathlib import Path  
  
import numpy as np  
import torch  
import torch.nn as nn  
import torch.optim as optim
```

```

from torch.amp import autocast, GradScaler
from torch.optim.swa_utils import AveragedModel, get_ema_multi_avg_fn
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as T
from sklearn.model_selection import GroupShuffleSplit
from sklearn.preprocessing import LabelEncoder
from tqdm import tqdm
import timm
from PIL import Image

try:
    from skimage.color import rgb2hed, hed2rgb
    HAS_SKIMAGE = True
    print("scikit-image available – HED stain augmentation ENABLED")
except ImportError:
    HAS_SKIMAGE = False
    print("scikit-image not installed – HED augmentation DISABLED (pip
install scikit-image)")

print(f"PyTorch: {torch.__version__} | CUDA:
{torch.cuda.is_available()}")

```

## Task 1 — Configuration

All hyperparameters are stored per-model in `MODEL_CFG`. This allows training both DINOv2 and ResNet-50 in a single run with optimal settings for each, without any shared mutable global state.

```

KAGGLE_DATASET_SLUG = "ramachandraudupa/tcga-pathology-dataset"
DATA_DIR = Path("./data/tcga_train_data")
CHECKPOINT_DIR = Path("./data/checkpoints3")
LOG_FILE = Path("./data/train3.log")
MODELS_TO_TRAIN = ["dinov2_small", "resnet50"]
VAL_FRACTION = 0.15
SEED = 42
NUM_WORKERS = 16

BACKBONE = {
    "dinov2_small": "vit_small_patch14_reg4_dinov2",
    "resnet50": "resnet50",
}

MODEL_CFG = {
    "dinov2_small": {
        "lr": 1e-4, "weight_decay": 0.05, "l2rd_factor": 0.75,
        "warmup_epochs": 3, "epochs": 50, "batch_size": 256,
        "drop_path_rate": 0.1, "label_smoothing": 0.1,
        "grad_clip": 1.0, "ema_decay": 0.9998,
    },

```

```

    "resnet50": {
        "lr": 5e-4, "weight_decay": 0.01,
        "warmup_epochs": 3, "epochs": 50, "batch_size": 256,
        "drop_path_rate": 0.0, "label_smoothing": 0.1,
        "grad_clip": 1.0, "ema_decay": 0.9998,
    },
}
print("Config loaded.")

```

## Task 2 — Data Loading & Flat Label Construction

We read `manifest.csv` and construct a single flat label string per patch. The label encoder then maps these strings to integers 0..N-1.

```

def load_manifest(data_dir: Path):
    manifest_path = data_dir / "manifest.csv"
    paths, flat_labels = [], []
    missing = 0

    with open(manifest_path, newline="") as f:
        for row in csv.DictReader(f):
            cancer = row.get("cancer_type", "").strip()
            subtype = row.get("sample_type", "").strip()
            if not cancer or not subtype:
                continue
            filename = Path(row["patch_path"]).name
            local_path = data_dir / "pathology_data" / cancer /
filename
                if not local_path.exists():
                    missing += 1
                    continue
                # Flatten: "breast_cancer" + "Primary Tumor" ->
                "breast_cancer_Primary_Tumor"
                flat = f"{cancer}_{subtype.replace(' ', '_')}}"
                paths.append(str(local_path))
                flat_labels.append(flat)

    print(f"Loaded {len(paths)} valid samples ({missing} missing)")
    return paths, flat_labels

def patient_id(path: str) -> str:
    """Extract TCGA patient barcode from patch filename."""
    stem = Path(path).stem
    m = re.match(r"(TCGA-[A-Z0-9]{2})-[A-Z0-9]{4})", stem)
    return m.group(1) if m else stem.split("_x")[0]

print("Data loading functions defined.")

```

## Task 3 — HED Stain Augmentation

### Why HED and not just ColorJitter?

Standard `ColorJitter` perturbs RGB channels independently, which does not respect the physics of H&E staining. Hematoxylin (blue-purple, stains nuclei) and Eosin (pink, stains cytoplasm) are two **distinct dye channels** that interact in the RGB space.

**HED augmentation** (Tellez et al. 2019) works in the *correct* color space:

1. Decompose RGB  $\rightarrow$  HED (Hematoxylin, Eosin, DAB) using a fixed color deconvolution matrix
2. Randomly scale and shift each stain channel independently
3. Convert back to RGB

This simulates the actual variability in staining protocols across hospitals and scanners — the main driver of cross-patient domain shift. Training on these augmentations makes the model **stain-invariant** and closes the train/val gap.

RGB  $\rightarrow$  `rgb2hed()`  $\rightarrow$  [H, E, D]  $\rightarrow$  `scale * channel + shift`  $\rightarrow$  `hed2rgb()`  $\rightarrow$  RGB'

```
class HEDJitter:
    """Perturb H&E stain channels in HED color space (Tellez et al.
    2019)."""
    def __init__(self, sigma: float = 0.05, p: float = 0.8):
        self.sigma = sigma  # max  $\pm$  fractional perturbation per stain
        self.p = p

    def __call__(self, img):
        if not HAS_SKIMAGE or np.random.rand() > self.p:
            return img
        arr = np.asarray(img).astype(np.float32) / 255.0
        hed = rgb2hed(arr)
        alpha = np.random.uniform(1 - self.sigma, 1 + self.sigma,
        size=3) # multiplicative
        beta = np.random.uniform(-self.sigma, self.sigma,
        size=3) # additive
        hed = hed * alpha + beta
        rgb = np.clip(hed2rgb(hed), 0.0, 1.0)
        return Image.fromarray((rgb * 255.0).astype(np.uint8))

# Visualize HED augmentation effect
import matplotlib.pyplot as plt

if HAS_SKIMAGE:
    from skimage.color import rgb2hed, hed2rgb
    # Create a synthetic pink/purple patch resembling H&E tissue
```

```

patch = np.zeros((224, 224, 3), dtype=np.uint8)
patch[... , 0] = 200; patch[... , 1] = 150; patch[... , 2] = 190 #
pinkish-purple

jitter = HEDJitter(sigma=0.05, p=1.0)
original = Image.fromarray(patch)
augmented = jitter(original)

fig, axes = plt.subplots(1, 2, figsize=(8, 3))
axes[0].imshow(original); axes[0].set_title("Original");
axes[0].axis("off")
axes[1].imshow(augmented); axes[1].set_title("HED Jitter");
axes[1].axis("off")
plt.suptitle("Stain Augmentation in HED Color Space",
fontweight="bold")
plt.tight_layout(); plt.show()
else:
    print("Install scikit-image to see the HED augmentation demo.")

```

## Task 4 — Augmentation Pipeline

The full training transform pipeline, ordered to maximize augmentation effect while respecting the stain physics:

| Step | Transform                                       | Purpose                                                   |
|------|-------------------------------------------------|-----------------------------------------------------------|
| 1    | RandomResizedCrop(224, scale=0.7-1.0)           | Scale invariance                                          |
| 2    | RandomHorizontalFlip + RandomVerticalFlip       | Rotation invariance (tissue has no canonical orientation) |
| 3    | RandomRotation(90°) p=0.5                       | Full 90° rotation invariance                              |
| 4    | HEDJitter(sigma=0.05) p=0.8                     | <b>Stain-space augmentation</b> — key for domain shift    |
| 5    | ColorJitter(brightness=0.1, contrast=0.1) p=0.3 | Light residual RGB jitter                                 |
| 6    | GaussianBlur p=0.2                              | Scanner focus variation                                   |
| 7    | ToTensor + Normalize(ImageNet)                  | Convert to tensor                                         |
| 8    | RandomErasing(p=0.2)                            | Occlusion robustness                                      |

```

IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD  = [0.229, 0.224, 0.225]

def get_transforms(train: bool):
    if not train:
        return T.Compose([
            T.Resize((224, 224)),
            T.ToTensor(),

```

```

        T.Normalize(IMAGENET_MEAN, IMAGENET_STD),
    ])
    return T.Compose([
        T.RandomResizedCrop(224, scale=(0.7, 1.0), ratio=(0.85,
1.18)),
        T.RandomHorizontalFlip(),
        T.RandomVerticalFlip(),
        T.RandomApply([T.RandomRotation(90)], p=0.5),
        HEDJitter(sigma=0.05, p=0.8),                                # stain-
space jitter
        T.RandomApply([T.ColorJitter(brightness=0.1, contrast=0.1)],
p=0.3),
        T.RandomApply([T.GaussianBlur(kernel_size=3, sigma=(0.1,
1.5))], p=0.2),
        T.ToTensor(),
        T.Normalize(IMAGENET_MEAN, IMAGENET_STD),
        T.RandomErasing(p=0.20, scale=(0.02, 0.15)),
    ])

print("Augmentation pipeline defined.")

```

## Task 5 — Model Architecture

### DINOv2 ViT-S/14 (with Register Tokens)

DINOv2 is a **self-supervised vision transformer** pretrained on a curated dataset of 142M images using self-distillation. It produces significantly better visual features than supervised ImageNet pretraining — especially for fine-grained tasks like pathology, where meaningful visual patterns are subtle.

We use `vit_small_patch14_reg4_dinov2` — the ViT-Small variant with:

- **14×14 patch size** (finer spatial resolution than ViT-S/16)
- **4 register tokens** — additional [REG] tokens that absorb background artifacts and produce cleaner patch-level features (Darcet et al. 2023)

### Classification Head

A lightweight 3-layer head on top of the backbone CLS token:

```
LayerNorm(feat_dim) → Dropout(0.2) → Linear(feat_dim, num_classes)
```

LayerNorm before the linear layer stabilizes training by normalizing the feature distribution, which can vary significantly between pretraining and fine-tuning domains.

```

class SingleHeadModel(nn.Module):
    def __init__(self, model_name: str, num_classes: int,
drop_path_rate: float = 0.0):

```

```

    super().__init__()
    if model_name == "dinov2_small":
        self.backbone = timm.create_model(
            BACKBONE[model_name], pretrained=True, num_classes=0,
            img_size=224, drop_path_rate=drop_path_rate,
        )
    elif model_name == "resnet50":
        self.backbone = timm.create_model(
            BACKBONE[model_name], pretrained=True, num_classes=0,
            global_pool="avg",
        )
    else:
        raise ValueError(f"Unknown model '{model_name}'")

    feat_dim = self.backbone.num_features
    self.head = nn.Sequential(
        nn.LayerNorm(feat_dim),
        nn.Dropout(0.2),
        nn.Linear(feat_dim, num_classes),
    )
    print(f"{model_name}: feat_dim={feat_dim},
classes={num_classes}")

    def forward(self, x):
        return self.head(self.backbone(x))

# Demo with 10 dummy classes
for name in ["dinov2_small", "resnet50"]:
    m = SingleHeadModel(name, num_classes=10)
    out = m(torch.randn(2, 3, 224, 224))
    print(f" {name}: output shape = {out.shape}")

```

## Task 6 — Layer-wise Learning Rate Decay (LLRD) for DINOv2

Vision Transformers have 12 transformer blocks arranged in a hierarchy: **early blocks** learn low-level texture features, **late blocks** learn high-level semantic features.

When fine-tuning, we want:

- **Late blocks** → higher LR (closer to the task head, need more adaptation)
- **Early blocks** → lower LR (general features, risk of catastrophic forgetting)
- **Patch embedding / positional encoding** → very low LR (pretrained geometry)

LLRD applies a multiplicative decay factor (0.75) per block going from head to stem:

```

Block 11 (closest to head): LR × 0.750 = 1.00 × base_lr
Block 10:                    LR × 0.751 = 0.75 × base_lr

```

|                       |                                                      |
|-----------------------|------------------------------------------------------|
| Block 9:              | $LR \times 0.75^2 = 0.56 \times \text{base\_lr}$     |
| ...                   | ...                                                  |
| Block 0:              | $LR \times 0.75^{11} = 0.042 \times \text{base\_lr}$ |
| Patch embed / tokens: | $LR \times 0.75^{12} = 0.032 \times \text{base\_lr}$ |

No-decay groups (bias, LayerNorm) always get `weight_decay=0`.

```
# Visualize LLRD: show effective LR per block for DINOv2-small
import matplotlib.pyplot as plt

base_lr = 1e-4
llrd = 0.75
num_layers = 12

blocks = list(range(num_layers))
lrs = [base_lr * (llrd ** (num_layers - 1 - i)) for i in blocks]

plt.figure(figsize=(9, 3))
plt.bar([f"Block {i}" for i in blocks], lrs, color="steelblue")
plt.axhline(base_lr * (llrd ** num_layers), color="orange",
            linestyle="--", label="Patch embed")
plt.axhline(base_lr, color="green", linestyle="--", label="Head")
plt.ylabel("Learning Rate"); plt.title("Layer-wise LR Decay
(LLRD=0.75) for DINOv2-Small")
plt.xticks(rotation=45); plt.legend(); plt.tight_layout(); plt.show()
```

## Task 7 — Class-Weighted Loss (vs WeightedRandomSampler)

The TCGA dataset is imbalanced — some `cancer_type` + `sample_type` combinations are much rarer than others.

We now handle this with **class-weighted CrossEntropyLoss** instead of a `WeightedRandomSampler`:

- **Sampler:** changes which images you see (oversamples rare classes)
- **Weighted loss:** changes how much each image contributes to the gradient (upweights rare classes)

Using *both* would double-correct and hurt majority-class accuracy. We chose weighted loss because:

1. It is applied on the gradient level, not the sampling level — rare classes have stronger signal
2. The training loader can use plain shuffle, which has lower overhead than the sampler
3. It interacts better with batch normalization (balanced sampler can distort BN statistics)

Weights are **`sqrt-inverse-frequency normalized to mean 1`** to keep the loss scale (and thus the LR) comparable to an unweighted baseline.

```

def compute_class_weights(tr_labels: np.ndarray, num_classes: int,
device: torch.device):
    counts = np.bincount(tr_labels,
minlength=num_classes).astype(np.float64)
    class_w = 1.0 / np.sqrt(np.maximum(counts, 1.0))
    class_w = class_w / class_w.mean() # normalize to mean=1
    return torch.as_tensor(class_w, dtype=torch.float, device=device)

# Demo: simulate imbalanced class distribution
np.random.seed(42)
demo_counts = np.array([5000, 500, 200, 50, 10]) # severely
imbalanced
demo_labels = np.repeat(np.arange(5), demo_counts)

weights = compute_class_weights(demo_labels, num_classes=5,
device=torch.device("cpu"))
print("Class counts :", demo_counts)
print("Class weights:", [f"{w:.3f}" for w in weights.numpy()])

import matplotlib.pyplot as plt
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 3))
ax1.bar(range(5), demo_counts, color="salmon"); ax1.set_title("Class
Frequency"); ax1.set_xlabel("Class")
ax2.bar(range(5), weights.numpy(), color="steelblue");
ax2.set_title("sqrt-Inv-Freq Weights (normalized)");
ax2.set_xlabel("Class")
plt.tight_layout(); plt.show()

```

## Task 8 — Slide-Level Evaluation

Individual patches from the same patient slide are **not independent** — they share the same tissue, stain, and scanner. A model that correctly classifies most patches from a slide but misclassifies a few noisy ones is still a good model.

**Slide-level aggregation** pools patch predictions per patient case:

1. For each validation patch, compute the softmax probability vector over all classes
2. **Sum** the probability vectors for all patches from the same patient case
3. The slide prediction = argmax of the summed probability vector
4. Compare against the slide's majority true label

This is called **soft voting** or **probability pooling**. It is more robust than hard voting (majority of argmax predictions) because soft probabilities carry confidence information.

**Why best checkpoint is selected on slide accuracy, not patch accuracy:**

Slide-level accuracy is the clinically relevant number — a pathologist evaluates a whole slide, not individual crops. Optimizing for slide accuracy better reflects the downstream use case.

```

# Illustrate slide-level aggregation with a synthetic example
import matplotlib.pyplot as plt

np.random.seed(0)
num_classes = 5
true_class = 2 # the actual label for this slide

# Simulate 20 patches from the same slide: mostly correct, a few noisy
patch_probs = []
for _ in range(18): # 18 confident correct patches
    p = np.random.dirichlet([0.1]*num_classes)
    p[true_class] += 1.5
    p = p / p.sum()
    patch_probs.append(p)
for _ in range(2): # 2 noisy wrong patches
    p = np.random.dirichlet([1.0]*num_classes)
    patch_probs.append(p)

patch_preds = [np.argmax(p) for p in patch_probs]
slide_pred = np.argmax(np.sum(patch_probs, axis=0))

from collections import Counter
hard_vote = Counter(patch_preds).most_common(1)[0][0]

print(f"True class      : {true_class}")
print(f"Hard vote pred    : {hard_vote} ({'✓' if hard_vote ==
true_class else 'x'})")
print(f"Soft prob pooling: {slide_pred} ({'✓' if slide_pred ==
true_class else 'x'})")

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 3))
ax1.bar(range(num_classes), np.sum(patch_probs, axis=0),
color="steelblue")
ax1.axvline(true_class, color="red", linestyle="--", label=f"True
class={true_class}")
ax1.set_title("Summed patch probabilities → slide prediction")
ax1.legend(); ax1.set_xlabel("Class")

patch_pred_counts = [patch_preds.count(c) for c in range(num_classes)]
ax2.bar(range(num_classes), patch_pred_counts, color="salmon")
ax2.axvline(true_class, color="red", linestyle="--", label=f"True
class={true_class}")
ax2.set_title("Patch-level argmax votes")
ax2.legend(); ax2.set_xlabel("Class")
plt.tight_layout(); plt.show()

```

## Task 9 — EMA (Exponential Moving Average)

An EMA model maintains a **smoothed copy** of model weights:

```
ema_weights = decay × ema_weights + (1 - decay) × model_weights
```

With `decay=0.9998`, the EMA averages over roughly the last **5,000 gradient steps** ( $\approx$  several epochs).

The EMA model is evaluated at every epoch alongside the live model. It consistently achieves higher validation accuracy because weight averaging smooths out the sharp minima and noisy updates that the live model experiences during training.

We maintain **separate best checkpoints** for the live model and EMA model, and select checkpoints based on **slide-level accuracy** rather than patch-level accuracy.

## Task 10 — Training Results & Metrics

The training loop logs all metrics to a CSV file and generates training curves at the end of each model's run.

```
import matplotlib.pyplot as plt
import pandas as pd

def plot_all_metrics(checkpoint_dir: Path):
    for model_name in ["dinov2_small", "resnet50"]:
        csv_path = checkpoint_dir / f"{model_name}_metrics.csv"
        if not csv_path.exists():
            print(f"No metrics found for {model_name}")
            continue

        df = pd.read_csv(csv_path)
        fig, axes = plt.subplots(1, 3, figsize=(18, 4))

        axes[0].plot(df["epoch"], df["tr_loss"], label="Train")
        axes[0].plot(df["epoch"], df["va_loss"], label="Val")
        axes[0].plot(df["epoch"], df["ema_va_loss"], label="EMA Val",
linestyle="--")
        axes[0].set_title("Loss"); axes[0].legend();
axes[0].grid(True)

        axes[1].plot(df["epoch"], df["tr_acc"], label="Train
(patch)")
        axes[1].plot(df["epoch"], df["va_acc"], label="Val
(patch)")
        axes[1].plot(df["epoch"], df["ema_va_acc"], label="EMA Val
(patch)", linestyle="--")
        axes[1].set_title("Patch Accuracy"); axes[1].legend();
axes[1].grid(True)

        if "va_slide_acc" in df.columns:
            axes[2].plot(df["epoch"], df["va_slide_acc"],
label="Val (slide)", linewidth=2)
            axes[2].plot(df["epoch"], df["ema_va_slide_acc"],
```

```

label="EMA Val (slide)", linestyle="--", linewidth=2)
    axes[2].set_title("Slide-Level Accuracy");
axes[2].legend(); axes[2].grid(True)

    plt.suptitle(f"{model_name} – Training Curves", fontsize=13,
fontweight="bold")
    plt.tight_layout(); plt.show()

plot_all_metrics(CHECKPOINT_DIR)

```

## Summary

| Component    | Design Choice                             | Rationale                                                                          |
|--------------|-------------------------------------------|------------------------------------------------------------------------------------|
| Label scheme | Flat <code>cancer_type_sample_type</code> | Simpler, avoids cascading gradient issues of dual-head                             |
| Backbone     | DINOv2 ViT-S/14-reg4 + ResNet-50          | DINOv2 self-supervised pretraining → better features for subtle pathology patterns |
| Imbalance    | Class-weighted CE loss (not sampler)      | Gradient-level correction; simpler loader                                          |
| Stain aug    | HED jitter ( $\sigma=0.05$ , $p=0.8$ )    | Directly addresses cross-patient domain shift from staining variability            |
| LLRD         | factor=0.75 for ViT                       | Prevents forgetting general features in early ViT layers                           |
| Evaluation   | Per-patch + per-slide accuracy            | Slide-level is the clinically relevant metric                                      |
| Best ckpt    | Slide-level accuracy                      | Closer to real-world deployment objective                                          |
| EMA          | decay=0.9998                              | Consistently beats live model; free ensemble                                       |

### To run training:

```

pip install timm scikit-image
python Train2.py

```

# Multi-Cancer Image Classification

## Transfer Learning on 47-Class Cancer Dataset | DINOv2 & ResNet-50

---

### Overview

This notebook documents the training pipeline for our **multi-cancer image classification** model, trained on the `ramachandraudupa/multicancer-dataset` — a curated collection of histopathology and medical imaging data spanning **47 distinct cancer categories**.

### What Makes This Dataset Different

Unlike the TCGA dataset (collected via the GDC API), this dataset was assembled by **curating images referenced across multiple cancer imaging research papers**. Each class corresponds to a distinct cancer type or tissue origin, and images were sourced from publicly released datasets that accompany peer-reviewed publications in computational pathology and oncology imaging.

The dataset follows a **standard ImageFolder layout** used universally in computer vision:

```
multicancer-dataset/  
  train/  
    ALL/           ← Acute Lymphoblastic Leukemia  
    BCC/           ← Basal Cell Carcinoma  
    Brain_Glioma/  
    Brain_Menin/  
    Brain_Tumor/  
    Breast_Cancer/  
    Cervical_Cancer/  
    Colon_Cancer/  
    ...           ← 47 total classes  
  val/  
    ALL/  
    BCC/  
    ...
```

The train/val split is **pre-made by the dataset** — we use it as-is without any additional splitting.

### Dataset Provenance — Research Papers Referenced

This dataset was assembled by reviewing cancer imaging datasets mentioned across several published research papers in computational pathology. The key sources and publications that informed the data collection include:

## Leukemia / Blood Cancers

- **ALL-IDB** (Acute Lymphoblastic Leukemia Image Database) — Labati et al., *ALL-IDB: The Acute Lymphoblastic Leukemia Image Database for Image Processing*, ICIP 2011

## Skin Cancers

- **ISIC Archive** — Codella et al., *Skin Lesion Analysis Toward Melanoma Detection: ISIC 2018 Challenge*, arXiv 2018
- **HAM10000** — Tschandl et al., *The HAM10000 dataset, a large collection of multi-source dermoscopic images of common pigmented skin lesions*, Scientific Data 2018

## Brain Tumors

- **Br35H** — Hamada, *Br35H: Brain Tumor Detection 2020*, Kaggle 2020
- **Figshare Brain Tumor Dataset** — Cheng et al., *Enhanced Performance of Brain Tumor Classification via Tumor Region Augmentation and Partition*, PLoS ONE 2015

## Colon & Lung Pathology

- **LC25000 (Lung and Colon Histopathological Image Dataset)** — Borkowski et al., *Lung and Colon Cancer Histopathological Image Dataset (LC25000)*, arXiv 2019

## Cervical Cancer

- **SIPaKMeD** — Plissiti et al., *SIPAKMED: A New Dataset for Feature and Image Based Classification of Normal and Pathological Cervical Cells*, ICIP 2018

## Kidney, Oral, Breast Pathology

- **BreakHis** — Spanhol et al., *A Dataset for Breast Cancer Histopathological Image Classification*, IEEE Trans. Biomed. Eng. 2016
- **TCGA-Kidney** — Clear Cell, Papillary, and Chromophobe subtypes via GDC
- **ORCA** (Oral Cancer Dataset) — referenced in oral cancer detection surveys

## Chest / Lung Imaging

- **ChestX-ray14** — Wang et al., *ChestX-ray8: Hospital-scale Chest X-ray Database and Benchmarks*, CVPR 2017

The combined dataset was uploaded to Kaggle as [ramachandraudupa/multicancer-dataset](#) for reproducibility.

## Task Classification vs TCGA Dataset

| Property   | TCGA Dataset (Train2)             | Multi-Cancer Dataset (Train3)                |
|------------|-----------------------------------|----------------------------------------------|
| Classes    | 32 (cancer type × sample type)    | 47 cancer categories                         |
| Image type | H&E histopathology patches        | Mixed: H&E, dermoscopy, X-ray, cytology      |
| Source     | TCGA via GDC API (EC2 pipeline)   | Research paper datasets (curated)            |
| Split      | Patient-level (GroupShuffleSplit) | Pre-made train/val folders                   |
| Task       | Hard (fine-grained subtype,       | Moderate (diverse cancer types, clear visual |

| Property      | TCGA Dataset (Train2)           | Multi-Cancer Dataset (Train3) |
|---------------|---------------------------------|-------------------------------|
| difficulty    | stain variability)              | separation)                   |
| Best accuracy | ~75% patch / higher slide-level | ~97% validation               |

The 47-class dataset is an **inter-cancer** classification task (different organs, very different morphology) which is intrinsically easier than TCGA's **intra-cancer** subtype task. The high accuracy is expected and reflects the visual distinctiveness of the classes, not data leakage.

## Dependencies & Imports

```
import os, sys, csv, json, time, logging, subprocess
from collections import deque
from pathlib import Path

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.amp import autocast, GradScaler
from torch.optim.swa_utils import AveragedModel, get_ema_multi_avg_fn
from torch.utils.data import Dataset, DataLoader,
WeightedRandomSampler
import torchvision.transforms as T
from sklearn.preprocessing import LabelEncoder
from tqdm import tqdm
import timm
from PIL import Image, ImageFile

# Allow slightly truncated images (some files in large datasets are
# cut off)
ImageFile.LOAD_TRUNCATED_IMAGES = True

print(f"PyTorch: {torch.__version__} | CUDA:
{torch.cuda.is_available()}")
```

## Task 1 — Configuration

```
KAGGLE_DATASET_SLUG = "ramachandraudupa/multicancer-dataset"
DATA_DIR = Path("./data/multicancer_data")
CHECKPOINT_DIR = Path("./data/checkpoints_multicancer")
LOG_FILE = Path("./data/train_multicancer.log")

MODELS_TO_TRAIN = ["dinov2_small", "resnet50"]
SEED = 42
NUM_WORKERS = 16
IMG_EXTS = {".jpg", ".jpeg", ".png", ".bmp", ".tif", ".tiff",
".webp"}
```

```

BACKBONE = {
    "dinov2_small": "vit_small_patch14_reg4_dinov2",
    "resnet50": "resnet50",
}

MODEL_CFG = {
    "dinov2_small": {
        "lr": 1e-4, "weight_decay": 0.05, "lrd_factor": 0.75,
        "warmup_epochs": 3, "epochs": 50, "batch_size": 256,
        "drop_path_rate": 0.1, "label_smoothing": 0.1,
        "grad_clip": 1.0, "ema_decay": 0.9998,
    },
    "resnet50": {
        "lr": 5e-4, "weight_decay": 0.01,
        "warmup_epochs": 3, "epochs": 50, "batch_size": 256,
        "drop_path_rate": 0.0, "label_smoothing": 0.1,
        "grad_clip": 1.0, "ema_decay": 0.9998,
    },
}

print("Configuration loaded.")

```

## Task 2 — Auto-Detecting the Dataset Layout

Kaggle datasets are downloaded as a zip that extracts into a folder whose name is not always predictable. The `find_split_dirs()` function performs a **BFS search** up to 3 levels deep to locate any folder named `train/training` paired with `val/valid/validation`, making the code robust to different zip extraction structures.

```

./data/multicancer_data/
  Cancer_Dataset/           ← unknown top-level folder name
    train/                  ← found by BFS
      ALL/
      BCC/
      ...
    val/                    ← found by BFS
      ALL/
      ...

def find_split_dirs(root: Path, max_depth: int = 3):
    """BFS to find (train_dir, val_dir) anywhere within root, up to
    max_depth."""
    if not root.exists():
        return None

    train_names = {"train", "training"}
    val_names   = {"val", "valid", "validation", "test"}

    queue = deque([(root, 0)])
    while queue:

```

```

        cur, depth = queue.popleft()
        try:
            subdirs = [d for d in cur.iterdir() if d.is_dir()]
        except (PermissionError, FileNotFoundError):
            continue

        by_name = {d.name.lower(): d for d in subdirs}
        train_dir = next((by_name[n] for n in train_names if n in
by_name), None)
        val_dir = next((by_name[n] for n in val_names if n in
by_name), None)
        if train_dir and val_dir:
            return train_dir, val_dir

        if depth < max_depth:
            queue.extend((d, depth + 1) for d in subdirs)
    return None

# Test on the actual data directory if it exists
result = find_split_dirs(DATA_DIR)
if result:
    train_dir, val_dir = result
    print(f"Train dir: {train_dir}")
    print(f"Val dir: {val_dir}")
    print(f"Train classes: {len(list(train_dir.iterdir()))}")
    print(f"Val classes: {len(list(val_dir.iterdir()))}")
else:
    print("Dataset not yet downloaded. Run the training script to
download automatically.")

```

## Task 3 — Scanning the Dataset Structure

The `scan_split()` function walks each class sub-directory and collects all image paths, assigning the **folder name as the class label**. This supports any number of classes without hardcoding — adding a new cancer type to the dataset requires no code changes.

```

def scan_split(split_dir: Path):
    """Scan a split directory. Each subdirectory = one class."""
    paths, label_names = [], []
    class_dirs = sorted([d for d in split_dir.iterdir() if
d.is_dir()], key=lambda d: d.name)

    for cls_dir in class_dirs:
        n_before = len(paths)
        for img in cls_dir.rglob("*"):
            if img.is_file() and img.suffix.lower() in IMG_EXTS:
                paths.append(str(img))
                label_names.append(cls_dir.name)

```

```

        if len(paths) == n_before:
            print(f" WARNING: no images in class folder:
{cls_dir.name}")

        return paths, label_names

# Visualize class distribution if data is present
import matplotlib.pyplot as plt
from collections import Counter

if result:
    tr_paths, tr_names = scan_split(train_dir)
    va_paths, va_names = scan_split(val_dir)

    enc = LabelEncoder().fit(sorted(set(tr_names) | set(va_names)))
    num_classes = len(enc.classes_)

    train_counts = Counter(tr_names)
    classes_sorted = sorted(train_counts.keys())

    fig, ax = plt.subplots(figsize=(16, 4))
    ax.bar(range(len(classes_sorted)), [train_counts[c] for c in
classes_sorted], color="steelblue")
    ax.set_xticks(range(len(classes_sorted)))
    ax.set_xticklabels(classes_sorted, rotation=90, fontsize=8)
    ax.set_title(f"Train Set Class Distribution – {num_classes}
Classes")
    ax.set_ylabel("Number of Images")
    plt.tight_layout(); plt.show()

    print(f"Total train images : {len(tr_paths)}")
    print(f"Total val images   : {len(va_paths)}")
    print(f"Classes detected   : {num_classes}")
else:
    print("Dataset not yet downloaded.")

```

## Task 4 — Robust Image Loading

Real-world datasets from multiple sources often contain a small fraction of **corrupt, truncated, or misnamed files** (e.g., a file named `.jpg` that is actually an HTML error page from a failed download).

Two protections are in place:

1. `ImageFile.LOAD_TRUNCATED_IMAGES = True` — allows PIL to decode JPEGs that are slightly cut off at the end (most common case).
2. `__getitem__` wraps the decode in a `try/except OSError` — if a file truly cannot be decoded, it logs a warning and loads the next valid sample instead of crashing the entire training run.

```

class PatchDataset(Dataset):
    def __init__(self, paths, labels, transform=None):
        self.paths = paths
        self.labels = labels
        self.transform = transform

    def __len__(self):
        return len(self.paths)

    def __getitem__(self, idx):
        # Try idx first; on failure, scan forward for a readable image
        for offset in range(len(self.paths)):
            j = (idx + offset) % len(self.paths)
            try:
                img = Image.open(self.paths[j]).convert("RGB")
            except (OSError, Image.DecompressionBombError) as e:
                if offset == 0:
                    print(f"Unreadable image skipped: {self.paths[j]} ({e})")
                continue
            if self.transform:
                img = self.transform(img)
            return img, int(self.labels[j])
        raise RuntimeError("No readable images found in dataset.")

print("PatchDataset with robust loading defined.")

```

## Task 5 — Augmentation Pipeline

Because the 47 classes include diverse imaging modalities (H&E patches, dermoscopy, cytology slides, X-rays), we use a **general-purpose augmentation strategy** rather than the domain-specific HED stain augmentation from Train2.

| Augmentation                      | Rationale                                  |
|-----------------------------------|--------------------------------------------|
| RandomResizedCrop                 | Scale/translation invariance               |
| RandomHorizontal/<br>VerticalFlip | Tissue orientation-invariant               |
| RandomRotation(90°)               | Cancer cells have no canonical orientation |
| ColorJitter                       | Stain variability across labs and scanner  |
| GaussianBlur                      | Scanner focus variation                    |
| RandomErasing                     | Occlusion robustness (artifact patches)    |

```

IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD  = [0.229, 0.224, 0.225]

```

```

def get_transforms(train: bool):
    if not train:
        return T.Compose([
            T.Resize((224, 224)),
            T.ToTensor(),
            T.Normalize(IMAGENET_MEAN, IMAGENET_STD),
        ])
    return T.Compose([
        T.RandomResizedCrop(224, scale=(0.7, 1.0), ratio=(0.85,
1.18)),
        T.RandomHorizontalFlip(),
        T.RandomVerticalFlip(),
        T.RandomApply([T.RandomRotation(90)], p=0.5),
        T.RandomApply([
            T.ColorJitter(brightness=0.2, contrast=0.2,
saturation=0.2, hue=0.05)
        ], p=0.5),
        T.RandomApply([T.GaussianBlur(kernel_size=3, sigma=(0.1,
1.5))], p=0.2),
        T.ToTensor(),
        T.Normalize(IMAGENET_MEAN, IMAGENET_STD),
        T.RandomErasing(p=0.20, scale=(0.02, 0.15)),
    ])

print("Augmentation pipeline defined.")

```

## Task 6 — Model Architecture

Same architecture as Train2 — the **number of output classes is determined at runtime** from the dataset's folder structure, not hardcoded. This means the same code works for 47 classes here or 32 classes in the TCGA dataset.

```

class SingleHeadModel(nn.Module):
    def __init__(self, model_name: str, num_classes: int,
drop_path_rate: float = 0.0):
        super().__init__()
        if model_name == "dinov2_small":
            self.backbone = timm.create_model(
                BACKBONE[model_name], pretrained=True, num_classes=0,
                img_size=224, drop_path_rate=drop_path_rate,
            )
        elif model_name == "resnet50":
            self.backbone = timm.create_model(
                BACKBONE[model_name], pretrained=True, num_classes=0,
                global_pool="avg",
            )
        else:
            raise ValueError(f"Unknown model '{model_name}'")

```

```

        feat_dim = self.backbone.num_features
        self.head = nn.Sequential(
            nn.LayerNorm(feat_dim),
            nn.Dropout(0.2),
            nn.Linear(feat_dim, num_classes),
        )
        print(f"{model_name}: feat_dim={feat_dim} → {num_classes} classes")

    def forward(self, x):
        return self.head(self.backbone(x))

# Demo with 47 classes
m = SingleHeadModel("resnet50", num_classes=47)
out = m(torch.randn(2, 3, 224, 224))
print(f"Output shape: {out.shape} (batch=2, classes=47)")

```

## Task 7 — Class Imbalance: WeightedRandomSampler

Unlike Train2 (which uses class-weighted loss), Train3 uses `WeightedRandomSampler`. Both approaches correct for imbalance, but for a 47-class dataset where the classes are visually very distinct, the sampler is appropriate and simpler to reason about.

```

def make_sampler(tr_labels: np.ndarray, num_classes: int):
    counts = np.bincount(tr_labels, minlength=num_classes)
    class_w = 1.0 / np.sqrt(np.maximum(counts, 1))
    sample_weights = torch.as_tensor(class_w[tr_labels],
    dtype=torch.float)
    return WeightedRandomSampler(sample_weights, len(sample_weights),
    replacement=True)

print("Sampler helper defined.")

```

## Task 8 — Full Training Loop Summary

The training follows the same structure as Train2:

1. **Cosine LR + linear warmup** (3 epoch warmup, then cosine decay over 50 epochs)
2. **AMP** (mixed precision with `autocast` + `GradScaler`) — ~2× speedup
3. **Gradient clipping** at 1.0
4. **EMA model** (decay=0.9998) — evaluated in parallel, separate best checkpoint
5. **LLRD** for DINOv2 (layer-wise LR decay, factor=0.75)
6. **Crash-safe resume** — latest checkpoint saved every epoch
7. **CSV logging** — all metrics written per epoch for analysis

## Task 9 — Results & Visualizations

```
import matplotlib.pyplot as plt
import pandas as pd

def plot_all_metrics(checkpoint_dir: Path):
    for model_name in ["dinov2_small", "resnet50"]:
        csv_path = checkpoint_dir / f"{model_name}_metrics.csv"
        if not csv_path.exists():
            print(f"No metrics found for {model_name} – run Train3.py
first.")
            continue

        df = pd.read_csv(csv_path)
        fig, axes = plt.subplots(1, 2, figsize=(14, 4))

        axes[0].plot(df["epoch"], df["tr_loss"], label="Train")
        axes[0].plot(df["epoch"], df["va_loss"], label="Val")
        axes[0].plot(df["epoch"], df["ema_va_loss"], label="EMA Val",
linestyle="--")
        axes[0].set_title("Loss"); axes[0].legend();
axes[0].grid(True)
        axes[0].set_xlabel("Epoch")

        axes[1].plot(df["epoch"], df["tr_acc"], label="Train")
        axes[1].plot(df["epoch"], df["va_acc"], label="Val")
        axes[1].plot(df["epoch"], df["ema_va_acc"], label="EMA Val",
linestyle="--")
        axes[1].set_title("Accuracy"); axes[1].legend();
axes[1].grid(True)
        axes[1].set_xlabel("Epoch")

        best_val = df["va_acc"].max()
        best_ema = df["ema_va_acc"].max()
        plt.suptitle(
            f"{model_name} – Best Val: {best_val:.4f} Best EMA:
{best_ema:.4f}",
            fontsize=12, fontweight="bold"
        )
        plt.tight_layout(); plt.show()

plot_all_metrics(CHECKPOINT_DIR)
```

## Task 10 — Label Map Inspection

After training, a `label_map.json` is saved with the integer → class name mapping. This is required at inference time to convert model outputs back to human-readable labels.

```

label_map_path = CHECKPOINT_DIR / "label_map.json"

if label_map_path.exists():
    with open(label_map_path) as f:
        label_map = json.load(f)

    print(f"Total classes: {len(label_map)}")
    print("\nClass index → Label name:")
    for idx, name in sorted(label_map.items(), key=lambda x:
int(x[0])):
        print(f" {int(idx):>3} {name}")
else:
    print("label_map.json not found – run Train3.py first.")

```

## Summary

| Component      | Choice                                                            | Rationale                                                                                             |
|----------------|-------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Dataset        | ramachandraudupa/<br>multicancer-<br>dataset                      | 47 cancer types from multiple<br>research paper datasets                                              |
| Dataset split  | Pre-made train/ and<br>val/ folders                               | Curated by dataset author; BFS<br>auto-detection handles any<br>folder structure                      |
| Classes        | Detected automatically<br>from folder names                       | Code is dataset-agnostic —<br>works for any ImageFolder<br>layout                                     |
| Backbones      | DINOv2 ViT-S/14-reg4,<br>ResNet-50                                | Strong pretrained features for<br>diverse imaging modalities                                          |
| Imbalance      | WeightedRandomSam-<br>pler                                        | Oversamples rare cancer types<br>during training                                                      |
| Augmentation   | Rotation, flip,<br>ColorJitter,<br>GaussianBlur,<br>RandomErasing | General-purpose; no stain-<br>specific augmentation since<br>classes include X-rays and<br>dermoscopy |
| EMA            | decay=0.9998                                                      | Better validation accuracy<br>through weight averaging                                                |
| Robust loading | LOAD_TRUNCATED_IM-<br>AGES + fallback in<br>__getitem__           | Handles corrupt files from multi-<br>source dataset assembly                                          |

### To run training:

```

pip install timm
python Train3.py

```

The script automatically downloads the dataset from Kaggle, scans the folder structure, trains both models sequentially, and pushes results to `ramachandraudupa/multicancer-results-transfer`.